v Publications using TU Darmstadt's Corporate Design (TUDaCI)-0 <00

A Learned Advisor for Automated Materialized View Selection

Master thesis by Michael Girges
Date of submission: November 30, 2025

1. Review: Prof. Dr. Carsten Binnig
2. Review: Johannes Wehrstein
3. Review: Roman Heinrich
Darmstadt

TECHNISCHE
UNIVERSITÄT
DARMSTADT

Computer Science
Department
Data Management Lab

Erklärung zur Abschlussarbeit gemäß §22 Abs. 7 und §23 Abs. 7 APB der TU Darmstadt

Hiermit versichere ich, Michael Girges, die vorliegende Masterarbeit ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Alle Stellen, die Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Fall eines Plagiats (§38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei der abgegebenen Thesis stimmen die schriftliche und die zur Archivierung eingereichte elektronische Fassung gemäß §23 Abs. 7 APB überein.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 30. November 2025

Signature

Abstract

Materialized View (MV)s are commonly used in databases to accelerate read queries, but their benefits come with high maintenance costs and full refreshes to avoid stale data. Incremental Materialized View (IMV)s offer an alternative by updating views incrementally through triggers, yet they introduce significant write-time overhead that varies based on workload characteristics. This work proposes a machine-learning-based advisor that, given a SQL workload, predicts whether to use MV, IMV, or no views at all for optimal performance. To predict the cost of view maintenance, we introduce a modified decision-tree cost model inspired by T3; unlike standard per-tuple approaches, our model utilizes context features to predict total view refresh time, allowing it to capture the non-linear overhead of maintenance triggers. While the current approach requires a staging phase to capture internal trigger plans, the resulting recommendations yield significant performance gains. Evaluation on an unseen IMDB workload demonstrates that the advisor achieves a 3.86x speedup in total workload runtime compared to a full-materialization baseline. Furthermore, while zero-shot transfer limits strict classification accuracy, the advisor achieves 91.67% accuracy within a 25% tolerance threshold, successfully avoiding performance degradation by filtering out expensive view configurations.

Contents

List of Figures

List of Abbreviations

API Application Programming Interface

DBMS Database Management System

MV Materialized View

IMV Incremental Materialized View

IVM Incremental View Maintenance

DML Data Manipulation Language

ML Machine Learning

ILP Integer Linear Programming

DTA Database Tuning Advisor

T3 Tuple Time Tree

QEP Query Execution Plan

1. Introduction

1.1. Context and Motivation

MVs are a common optimization in database systems: by precomputing and storing query results they can significantly accelerate read queries. However, materialized views introduce maintenance costs whenever base tables change, because the view contents must be kept consistent with the underlying data. The naive strategy of full recomputation can become prohibitively expensive for workloads with frequent updates.

Incrementally maintained materialized views (IMVs) provide an alternative maintenance strategy by applying only the changes (deltas) caused by updates to base tables, instead of recomputing entire views. IMVs can significantly reduce maintenance work for many workloads, but they are not a universal win: their benefit depends on workload characteristics. In particular, IMVs tend to be most advantageous for read-heavy workloads with relatively small or structured updates, while write-heavy workloads can cause high incremental maintenance overhead and complex update propagation.

Choosing between no views, conventional MVs, and IMVs therefore requires trading off read-performance gains against the cost of maintaining views under updates. Manually exploring this trade-off is time-consuming and error-prone. This thesis proposes to automate that decision by building a machine learning-based advisor that, given a SQL workload composed of SELECT and Data Manipulation Language (DML) statements, recommends the view strategy that is likely to be optimal in terms of overall system performance and maintenance cost.

1.2. Problem Statement

The central problem addressed in this work is:

> Given the estimated execution plans of a SQL workload (a sequence of SELECT and DML statements), determine whether the workload is optimally served by (i) no views, (ii) MVs with full recomputation, or (iii) IMVs with incremental maintenance, taking into account both query performance and view maintenance costs.

Concretely, this requires (a) modelling the costs and benefits of each strategy for a given workload, and (b) designing a predictor that maps workload characteristics to the recommended strategy. To support cost-aware decision making, this thesis includes the design of a view-refresh-cost estimation model inspired by decision-tree approaches (cf. the T3 system [15]).

This thesis answers the following research questions:

1. When is it beneficial to use MV, IMV, or no materialized view at all for a given SQL workload?
2. Can a machine learning model accurately and efficiently predict the optimal view strategy for a workload based on its read and write characteristics?

1.3. Goals and Contributions

The goals of this thesis are to design, implement, and evaluate an automated advisor that recommends one of the three view modes (no view, MV, or IMV) for a given SQL workload. The primary contributions are:

- A cost estimation component for view refresh operations that informs the advisor; this component leverages a decision-tree-based approach inspired by the T3 system[15].
- The design of a machine learning-based advisor that takes workload features as input and outputs a recommended view mode (no view / MV / IMV).
- An empirical evaluation demonstrating the advisor's effectiveness at selecting strategies that balance read performance and maintenance overhead across a variety of workloads.

Note that, due to current limitations in the underlying database system's support for IMVs, the Query Execution Plan (QEP)s of internal Incremental View Maintenance (IVM) triggers cannot be obtained using the standard `EXPLAIN` command. To circumvent this, the view-refresh-cost model is trained and evaluated using the actual execution times of maintenance operations, rather than estimated costs from query plans.

1.4. Thesis Outline

The remainder of the thesis is organized as follows:

- Chapter 2 presents background material and related work on materialized views, incremental maintenance techniques, workload-aware tuning approaches and cost estimation methods.
- Chapter 3 outlines the overall methodology, including data collection, feature engineering, view refresh modelling procedures and view advisor design.
- Chapter 4 reports the experimental evaluation of both the view-refresh cost model and the proposed view advisor. The evaluation includes setup, metrics, results, and analysis.
- Chapter 5 concludes the thesis, summarizes findings, and outlines directions for future work.

2. Background

2.1. Automated View Selection

Automated database tuning, notably the selection and management of physical design structures such as MVs and indexes, has long been a critical and challenging task for database administrators. Traditional approaches often rely on manual expertise [23] or on complex rule-based and enumeration-based heuristics, which become particularly cumbersome for rich MV structures or complex queries [1].

Early automated MV selection work explored heuristic systems that choose candidate views based on estimated cost and validate them against the actual database [1]. Commercial systems like Microsoft SQL Server's Database Tuning Advisor (DTA) provide workload-driven recommendations for indexes and MVs [1]. Other research formulates view selection as an optimization problem (for example, Integer Linear Programming (ILP)) [23]; while these formulations can be precise, they are often computationally expensive for large sets of subqueries or extensive workloads, and iterative methods may not guarantee convergence.

More recently, Machine Learning (ML) methods have been applied to a variety of database tasks with the goal of replacing handcrafted components by learned models that can automatically handle complexity and variability [7, 10]. A common paradigm in learned database components is workload-driven learning, which typically requires executing representative workloads on the target database to collect training data. Although effective in many settings, this data-collection step is costly and must often be repeated for each new database or substantially changed workload [10]. To address this limitation, zero-shot learning approaches have emerged: models are pretrained once and designed to generalize to unseen databases or workloads without per-database retraining. Initial zero-shot work has focused on tasks such as cost estimation [18], and researchers are investigating extensions of this paradigm to tasks like MV selection.

The maintenance of MVs - especially incremental maintenance - is another important axis in the literature. For example, commercial systems include refresh/maintenance modules that track refresh methods and schedules; some research and product directions explore using ML to predict refresh behavior or maintenance cost and to incorporate those predictions into recommendation logic [1]. This line of work suggests that predicted maintenance cost (including incremental refresh cost) can and should influence which views are recommended. The interaction between specific maintenance strategies (e.g., incremental refresh) and ML-based recommendation processes remains an active area where further research can provide valuable insights.

2.2. Query Performance Prediction Using Cost Models

Accurate query performance prediction is essential for effective physical design tuning, including MV selection. Traditional cost models, such as those used in query optimizers, estimate query execution time based on factors like I/O costs, CPU costs, and network costs [16]. However, these models often struggle to capture the complexities of modern database systems and workloads, leading to inaccuracies in performance predictions. The first cost model was proposed by Listgarten et al. [12] as part of their work on main memory databases. Moreover, a framework for building cost models for hierarchical database systems was introduced by Manegold et al.[13].

In the last couple of decades, learning-based techniques have been increasingly applied to the problem of query performance prediction. Gupta et al. pioneered the use of machine learning for query performance prediction by introducing a decision tree-based approach to predict query execution time ranges[5]. Query performance prediction has also been approached for static and dynamic workloads using regression models [2], who implemented these techniques on top of PostgreSQL. Calibration of existing cost models in PostgreSQL has been explored by Wu et al. [19], where they used small calibration workloads to adjust the cost model parameters. In combination with an analytical model, they were able to predict the query execution time for larger workloads[20]. This approach proved competitive to other contemporary learning-based methods.

Neural networks and deep learning techniques have also been employed for query performance prediction through feeding the tree-structured query plans into neural network architectures. For instance, Sun et al. [18] concluded that combining tree nodes is effectively handled using a pooling architecture. Zhou et al. proposed encoding single and multiple concurrent queries using a graph embedding approach [25]. This approach captures the interactions between concurrent queries and enables accurate predictions of execution times of whole workloads. Additionally, transformer-based architectures and augmenting the input data with database statistics have been shown to further improve prediction accuracy [24].

Since workload-based models do not work well in zero-shot settings, recent research has focused on developing zero-shot query performance prediction models. The first zero-shot query performance prediction model was proposed by Hilprecht et al. [6]. Their zero-shot model was trained and evaluated on different databases and workloads, demonstrating the ability to generalize to unseen databases without retraining. More recently, Wu et al. [21] introduced a zero-shot model that is trained on the complete Amazon Redshift [3] workload. They propose a hierarchical model in an effort to improve latency.

Learning-based approaches have shown promising results in query performance prediction, but they are often criticized in literature for their high end-to-end latency. This latency can be a significant drawback in scenarios where quick predictions are essential for decision-making. Lehmann et al. [11] commented on this issue by concluding that learning-based query optimizers have high optimization times, which renders them impractical for real-world usage. One approach to mitigate this issue is proposed by Wu et al. [21], where they use a query cache with a fallback to a local decision tree model to reduce the end-to-end latency of their zero-shot model.

Other non-learning-based approaches have been proposed to solve the query performance prediction problem. Since these approaches are not dependent on neural networks, they can be more interpretable and can be quicker to train and evaluate. For instance, cost models based on differential programming have been proposed by Hilprecht et al. [8]. They depend on learning some of the constants in

pre-existing cost model functions. Another approach is to use different groups of calibration constants for the fixed cost functions in the PostgreSQL cost model [22].

2.2.1. Tuple Time Tree (T3) Cost Model

Rieger et al. proposed the Tuple Time Tree (T3) cost model [15], which bridges the gap between high-accuracy, high-latency learning-based models and their lower-accuracy, low-latency counterparts. T3 is a hybrid cost model that combines a low-latency decision tree with pipeline-based query plan presentation and tuple-centric prediction targets.

The pipeline-based query plan representation divides the query plan into multiple pipelines, where each pipeline consists of a sequence of operators that can be executed without materializing intermediate results. This representation allows T3 to generate a flat feature vector for each pipeline and predicts its execution time individually. The total query execution time is calculated as the sum of all pipeline predictions.

On the other hand, tuple-centric prediction is based on the idea that, instead of predicting the total execution time for a pipeline, T3 predicts the expected time it takes to process a single tuple within that pipeline. Then, the per-tuple time is multiplied by the pipeline's input cardinality to estimate the total execution time of the pipeline. This approach is highly compatible with the decision tree architecture and is highly-scalable.

Furthermore, T3 improves inference time by compiling the trained model directly into native machine code using the `lleaves` framework. This step drastically reduces the average prediction latency from 22µs in interpreted trees and 1ms in zero-shot neural networks down to 4 microseconds.

While the standard T3 per-tuple prediction approach works well for read queries (where cost scales linearly with cardinality), it faces challenges when applied to IVM maintenance triggers. Maintenance operations often incur fixed overheads (e.g., transaction logging, latch contention) that do not scale linearly with the number of changed rows. Furthermore, when transferring models zero-shot to databases with vastly different scales (e.g., millions vs. billions of rows), per-tuple coefficients can lead to significant extrapolation errors. Consequently, in this work, we adapt the T3 architecture to predict *total execution time* directly, augmented by global context features, as detailed in Chapter 3.

2.3. Workload Generation

In this work, we require a diverse set of training and evaluation workloads to train and assess our proposed models. Workload generation is a well-researched area, with various methods proposed to create synthetic workloads that mimic real-world query patterns. For example, Kipf et al. [9] introduced a method for generating synthetic SQL queries based on schema information and sampling values from the underlying data. Hilprecht et al. [6] built on this approach by enabling the generation of more complex queries that might include joins, random index creation and predicates with null comparison, regex-based string comparison and disjunctions. Additionally, in the Master's thesis "Caching Strategies based on realistic Workloads," Martin Stemmer [17] investigates the practical, cost-based trade-off between full result caching and incremental updates in modern data warehouses.

2.4. Incremental View Maintenance and pg_ivm

While MVs significantly accelerate analytical queries by precomputing results, they introduce the overhead of view maintenance. As the underlying base tables are modified (inserted, updated, or deleted), the MV becomes stale. Traditional maintenance strategies in systems like PostgreSQL rely on a full refresh strategy, which recomputes the entire view definition from scratch. While simple to implement, full refresh is computationally expensive and inefficient for large datasets where only a small fraction of the data has changed.

2.4.1. Incremental View Maintenance (IVM)

IVM addresses the inefficiency of full refreshes by computing and propagating only the changes needed to bring the view up to date. Formally, given a view V derived from base relations $R_1, R_2, \ldots, R_n$ via a query function Q, and a set of changes ΔR_i applied to the base relations, IVM seeks to compute ΔV such that the new view state V' is:

$$V' = V \cup \Delta V^+ \setminus \Delta V^-$$

where ΔV^+ represents tuples to be added and ΔV^- represents tuples to be removed.

For views involving simple selection and projection, the delta calculation is straightforward. However, for views involving joins and aggregations, the maintenance logic becomes more complex. For joins, the distributive property is typically utilized. If $V = R \bowtie S$, and R changes by ΔR, the change to the view is calculated as $\Delta V = \Delta R \bowtie S$. For aggregations, IVM typically employs a counting algorithm [4]. To handle deletions correctly in a view with grouped aggregates (e.g., 'GROUP BY'), the system must track the multiplicity (count) of each group. When a tuple is deleted from the base table, the count for the corresponding group in the view is decremented; the group is removed from the view only when the count reaches zero.

2.4.2. pg_ivm Implementation

Standard PostgreSQL does not support IVM natively; it requires extensions or manual trigger-based setups. The `pg_ivm` extension [14] is a state-of-the-art solution that brings IVM capabilities to PostgreSQL. Unlike asynchronous approaches that might use logs to defer updates, `pg_ivm` implements immediate maintenance, where the view is updated synchronously within the same transaction that modifies the base table.

`pg_ivm` operates by automatically generating and attaching triggers to the base tables involved in the MV definition. When a DML operation occurs, two trigger operations are executed, as illustrated in Figure 2.1:

1. **Delta Calculation**: The triggers capture the transition tuples (old and new values) and compute the corresponding delta for the view using the differential update logic described in Section 2.4.1. The computed ΔV is stored in temporary structures local to the DML transaction.
2. **View Update**: The changes calculated by the first trigger are applied to the materialized view to keep it consistent with the base tables.

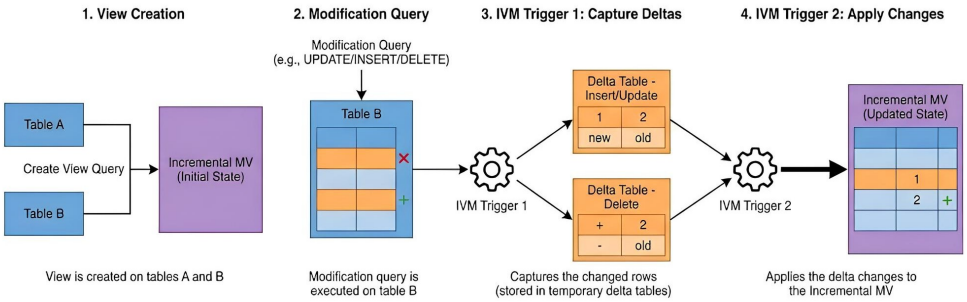

Figure 2.1.: pg_ivm Trigger Mechanism for Incremental View Maintenance

To support general SQL queries including joins and aggregations without requiring unique keys on all projected columns, `pg_ivm` maintains the view using "counting" logic similar to the classic Counting Algorithm. This ensures that views correctly reflect the underlying data multiplicity, particularly for ‘SUM‘ and ‘COUNT‘ aggregates.

It is also worth noting that, while `pg_ivm` drastically reduces the cost of reading fresh data compared to a full refresh, it imposes a write penalty on the base tables. Every insert or update to the base table now incurs the additional cost of computing ΔV and updating the MV index and storage. This trade-off—trading write latency for read consistency without full recomputation—makes `pg_ivm` a critical component in modern physical design tuning, as it alters the cost equations used by automated view selection algorithms. In some cases where the number of changed rows is a large portion of the total table cardinality, the incremental maintenance cost might be larger than a full refresh’s cost. Thus, `pg_ivm` provides a toggle to enable or disable IVM on a per-view basis, allowing users to choose between immediate incremental maintenance and traditional full refresh strategies based on their workload characteristics and performance requirements.

3. Methodology

This chapter details the methodology used to build the ML-based zero-shot view advisor. The primary objective is to create an advisor that can predict the optimal view strategy (no view, MV, or IMV) for a given SQL workload using execution time predictions of SELECT and DML execution plans from specialized T3 cost models. This is achieved by first curating a comprehensive training dataset by executing workloads and capturing their performance, then using this dataset to train a predictive cost model. The trained model is then integrated into a view advisor framework that analyzes workloads and recommends the best view configuration based on predicted costs and benefits.

3.1. Training Data Curation

The foundation of the ML advisor is a labeled dataset of query execution times. This dataset must capture performance metrics for both SELECT and DML statements in the IMV configuration. The phases mentioned below are repeated for 19 out of the 20 databases in the Amazon Redshift dataset [3], with the IMDB database being excluded to be used in evaluating the zero-shot model.

3.1.1. Dataset and Workload Generation

To build a robust model, a diverse set of training and evaluation workloads is required. We generated synthetic workloads that mimic the characteristics of real-world scenarios, such as the query distributions and read/write mixes found in datasets like the Amazon Redshift workload [3].

The data generation involves generating synthetic `SELECT` queries (read workload) using the parameters in Table A.1. Additionally, we generated synthetic DML statements (write workload) from a curated month-long snippet of the Amazon Redshift dataset using Stemmer's workload generator [17]. Both synthetic workloads' statements are verified syntactically. The workloads are also normalized by cleaning any database sharding suffixes in table names. Following that, we generated matches between the cleaned read and write workloads. The generated matches are divided into tiny workloads (also called "experiments") that consist of one SELECT statement and its corresponding DML queries. The structure of a single experiment is illustrated in Figure 3.1, while the overall workload structure is shown in Figure 3.2.

The matching between SELECT and DML queries is based on common base tables they operate on. In each experiment, the maximum number of queries is capped at 1 SELECT and 20 DML queries. The total number of queries in all generated experiments is capped at 100,000 queries. This is to ensure manageable execution times while still capturing sufficient variability.

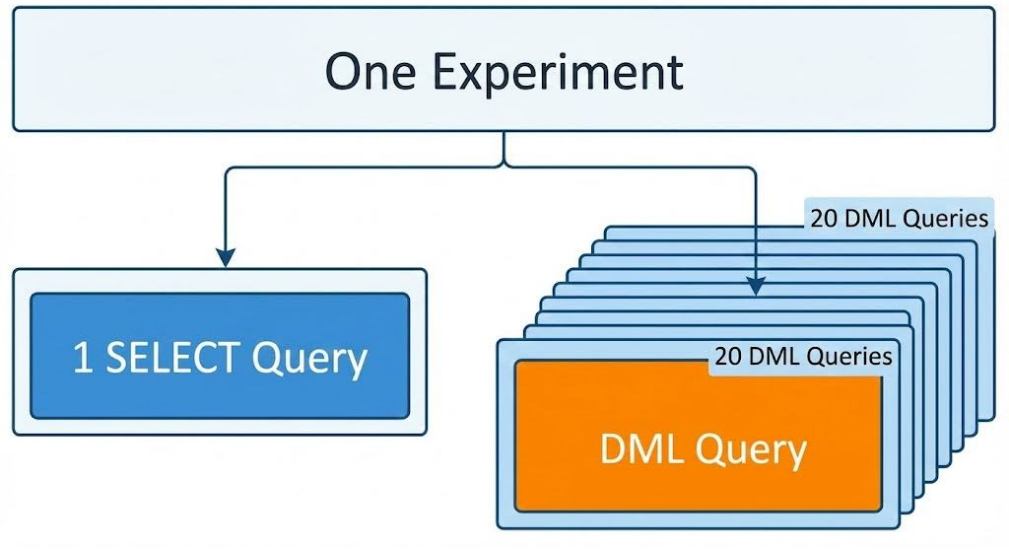

Figure 3.1.: Structure of a Single Experiment in the Workload

3.1.2. Experiment Data Collection

For each defined "experiment" workload, we executed the workload on the target database system under controlled conditions. This instrumentation is critical for capturing the ground-truth performance data.

Preparing the execution environment involves loading the schema and data, the creation of necessary indexes and loading of the necessary extensions–most importantly, the `pg_ivm` extension for IMV support. Staging tables are used to restore the database state between experiments to ensure consistency. The base tables are loaded as a 10% sample of the staging tables before each experiment execution. This is to ensure that the DML queries have data to operate on while avoiding conflicts with existing table rows. To avoid any cold start scenarios, the database is warmed up before execution. Each experiment is executed only in the IMV configuration to capture the relevant performance metrics for both reads and writes.

Each experiment execution consists of the following steps:

1. **Create the IMV for the read query:** For the read query in the experiment, we created an IMV. This sets up the necessary triggers and structures for incremental maintenance.
2. **Execute write queries and measure execution time:** Each DML query in the experiment is executed, and its execution time is measured. This captures the "trigger time overhead," which is the *view maintenance cost* incurred by the IMV triggers during writes.

This process produces a labeled dataset where each entry includes the query, its QEP, the view configuration, the measured execution time for the DML queries and the automatically executed IMV triggers mentioned in 2.4.2.

Note that the obtained trigger execution times only measure the "wall clock" time spent executing the specific SQL statements inside the triggers. They do not include any overhead from context switching, transaction management, or other database system activities outside the trigger execution itself. The total time spent on view maintenance (including overheads) is captured in the DML total execution

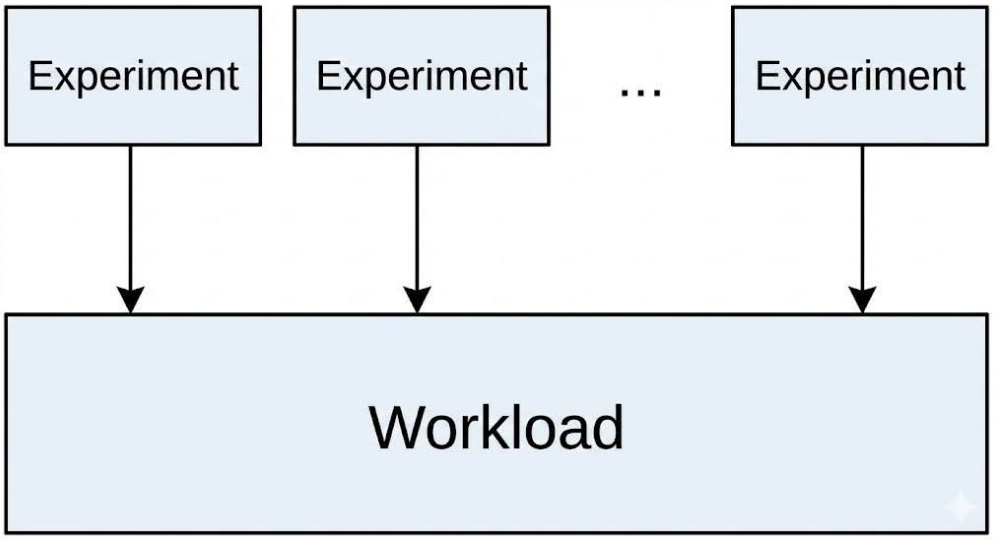

Figure 3.2.: Structure of the Entire Workload Composed of Multiple Experiments

time of the IVM triggers. This distinction is important for accurately training the cost model to predict the actual maintenance overhead.

3.1.3. Feature Extraction from Execution Plans

Once the raw performance data is collected, the next step is to parse the QEPs into numerical feature vectors. While the read-cost model utilizes the standard pipeline-centric feature extraction of the T3 system [15], the view refresh cost model requires a specialized approach to capture the overhead of IVM.

Standard query cost models typically predict execution time based solely on the operations within the specific plan. However, for IVM triggers, the cost is not just a function of the *operation* (the delta), but also of the *container* (the view size). A small update on a large view behaves differently than a small update on a small view. To capture this, we extracted a composite feature vector that combines three distinct sources of signal:

- **Delta Features:** These features are extracted from the IVM trigger execution plans (specifically the delta calculation and application steps). They characterize the dynamic volume of changes introduced by the DML statement. Key features include the estimated cost of the trigger logic (`delta_total_est_cost`), the number of rows being propagated (`delta_total_est_card`), and the count of specific operators like aggregations or scans within the trigger.
- **Context Features:** These features are extracted from the definition of the view itself (the original ‘SELECT‘ statement that created the view). They capture the static complexity and scale of the data structure being modified. Crucially, this includes `context_total_est_card` (the total cardinality of the view), `context_max_depth` (the depth of the view’s join tree), and the total number of operators. These features provide the necessary "state" information, allowing the model to distinguish between a low-cardinality update on a small lookup table versus a low-cardinality update on a billion-row aggregate view.

- **Interaction Features:** To enable zero-shot generalization across databases of vastly different sizes (e.g., training on the `tournament` database vs. testing on the `IMDB` database from Amazon Redshift), we engineered scale-invariant interaction ratios. The most critical feature is the *Impact Ratio*, which measures the relative magnitude of the update:

$$\text{Impact Ratio} = \frac{\text{Delta Rows}}{\text{View Cardinality} + 1}$$

  We added 1 to the denominator to avoid division by zero in case of zero-cardinality views. By explicitly exposing these ratios (along with others such as `context_scan_ratio` and `cost_density`), we prevented the tree-based model from overfitting to the specific absolute cardinalities seen during training, allowing it to extrapolate better to unseen schemas.

Prior to training, a featurized dataset is constructed by combining these composite vectors with their corresponding measured execution times. Data points where no IVM trigger fired (resulting in zero execution time) are filtered out to ensure the model focuses exclusively on learning the cost of actual maintenance work.

3.2. Model Training

With the featurized dataset, we trained the cost models. While a pre-existing read-cost model is reused, the development of a novel *view refresh cost model* to predict the IMV maintenance overhead, which is an integral part of the view advisor.

3.2.1. Training the T3-style Cost Model

We implemented the view refresh cost model as a LightGBM gradient-boosted decision-tree ensemble, an approach inspired by the T3 cost model [15]. The feature vectors extracted from the parsed trigger execution plans serve as the input (X) for this model. More specifically, for each DML QEP, the model is served the following data:

- The full set of pipeline features of the view's underlying query, extracted from the QEP of the view query itself.
- A subset of pipeline features extracted from the QEP of the delta computation trigger associated with the IMV.
- The full set of pipeline features from the QEP of the delta apply trigger associated with the IMV.

The training pipeline was adapted to handle the specific feature subset generated from these maintenance plans augmented by contextual information from the views query plans, ensuring the model learns the costs associated with incremental updates rather than general query execution.

3.2.2. Prediction Target and Transformation

While we retained the pipeline-centric architecture of the original T3 model (decomposing the query plan into segments), we altered the prediction target for the view refresh cost model.

The standard T3 approach predicts the *per-tuple* processing time for each pipeline and calculates the total cost by multiplying this prediction by the input cardinality. We demonstrated that standard tuple-centric cost models are insufficient for maintenance workloads due to fixed overheads, necessitating a total-time prediction approach augmented by context features. Specifically, assuming a constant per-tuple cost causes predictions to explode in value when transferring from training tables (e.g., 10^5 rows) to testing tables (e.g., 10^7 rows), as maintenance operations often incur fixed overheads or logarithmic scaling factors that are not captured by a linear per-tuple coefficient.

To address this, our model is trained to predict the **Total Execution Time** of each pipeline directly. By learning the absolute duration rather than a normalized rate, the tree model can utilize the *Context Features* (Section 3.1.3) to learn non-linear scaling laws appropriate for the database size. The final maintenance cost for a DML operation is derived by aggregating the predicted total times of its constituent pipelines:

$$\text{Cost}_{\text{maintenance}} = \sum_{p \in \text{Pipelines}} \text{Model}(x_p)$$

Furthermore, database maintenance times exhibit a heavy-tailed, Log-Normal distribution. To stabilize the gradient boosting process, we applied a `Log1p` transformation to the target variable t (measured in milliseconds) of each pipeline:

$$t' = \ln(1 + t)$$

This transformation compresses the dynamic range of the target, preventing the model from overfitting to extreme outliers while handling zero-cost pipelines gracefully. During training, outliers are clipped at the 99th percentile (P_{99}). At inference time, the model's output for each pipeline is reversed ($t = e^{t'} - 1$) before aggregation.

3.2.3. View Advisor Setup

The trained view refresh cost model is a key component of a larger view advisor system. The advisor is designed to recommend one of the following view configurations: IMVs, MVs or no view at all. The view advisor input is the estimate QEPs of a workload in each of the three view configurations. In case of IMV and MV, each estimate QEP of a `SELECT` statement contains the potential view estimated to be used. The advisor operates by estimating the effect of different view configurations on a given workload.

The setup of the view advisor involves the following steps:

1. **Workload Analysis:** The advisor takes the estimate QEPs of a given workload as input. This workload consists of a mix of read (`SELECT`) and DML queries. The estimate QEPs are generated for three configurations: no view, MV, and IMV.

2. **View Definitions:** A set of candidate views are created in the database beforehand by the user.
3. **Cost-Benefit Analysis:** For each candidate view, the advisor performs a cost-benefit analysis.
    - **Benefit:** The benefit is the estimated reduction in read query execution time. This is calculated by rewriting the workload queries to use the views and estimating the cost of the rewritten queries using a read-cost model.
    - **Cost:** The cost is the estimated overhead of maintaining the views. This is where our T3 view refresh cost model is used. For each DML query in the workload that affects a base table of the candidate view, the model predicts the maintenance cost.
    - **Total Score:** The total score for each view is computed as:

      $$\text{Total Score} = \text{Total Read Time with View} + \text{Total View Maintenance Cost}$$

      The advisor sums the predicted read times (using the read-cost model) and the predicted view maintenance costs (using the view refresh cost model) to get the total score for each view configuration.
4. **View Selection:** Based on the cost-benefit analysis, the advisor provides the following recommendations:
    - **Localized recommendations**: for each group of queries that share the same base table(s) as a candidate view, the advisor recommends whether to use the view (as MV or IMV) or not, based on which option yields the lowest total score.
    - **Global recommendation**: the advisor also provides a global recommendation across the entire workload, selecting the view configuration (no view, MV, or IMV) that minimizes the overall total score.

   This is typically formulated as an optimization problem, where the goal is to maximize the total benefit under a constraint on the total maintenance cost (a "maintenance budget").

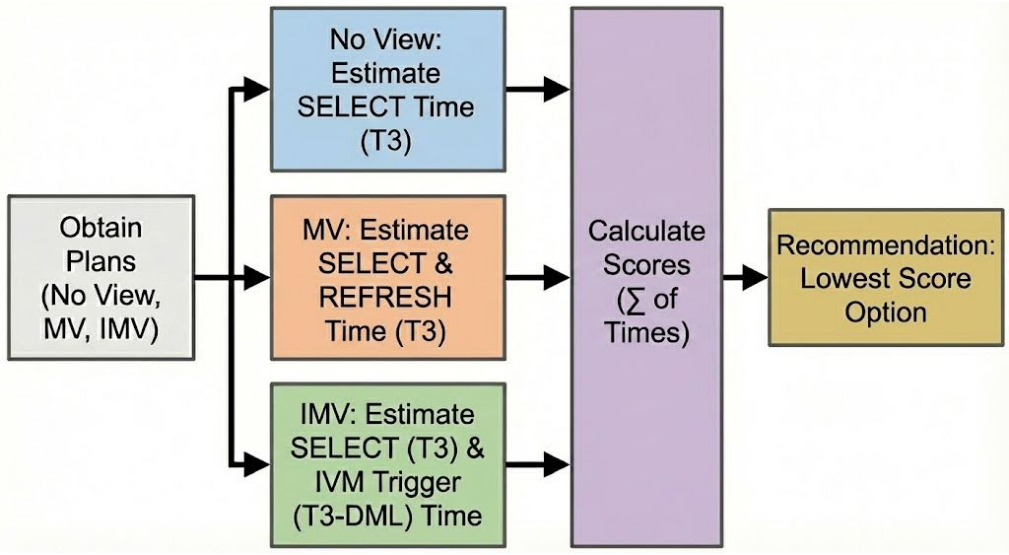

Figure 3.3.: View Advisor Cost-Benefit Analysis Flow

This setup allows the advisor to make informed decisions about view usage based on predicted performance impacts, leveraging the trained view refresh cost model to accurately estimate the overhead of IMV maintenance.

Note that, due to the PostgreSQL database system's current limitations in estimating IMVs internal triggers using `EXPLAIN`, the advisor currently operates in a "What-If" mode on a staging instance, where the actual execution times of maintenance operations are used instead of estimated costs.

3.3. Summary

This chapter outlined the comprehensive methodology for developing an ML-based zero-shot view advisor. The process began with the careful curation of a labeled dataset through controlled execution of synthetic workloads, capturing both read and write performance metrics in the IMV configuration. Feature extraction techniques were employed to transform QEPs into numerical representations suitable for model training. The core of the methodology involved training a novel T3-style view refresh cost model to predict IMV maintenance overhead. Finally, the trained model was integrated into a view advisor framework capable of recommending optimal view strategies based on cost-benefit analyses. This structured approach ensures that the advisor is grounded in empirical performance data and leverages advanced machine learning techniques to make informed recommendations.

4. Evaluation

To evaluate the performance and usefulness of the proposed ML-based view advisor, we conduct experiments on two key components: (1) the view refresh cost model, which predicts the maintenance overhead of IMVs, and (2) the view advisor itself, which recommends optimal view configurations for a given workload. This chapter presents the experimental setup, evaluation metrics, results, and analysis for both components.

4.1. Experimental Setup

This section describes the experimental environment, datasets, and procedures used to evaluate the view refresh cost model and the view advisor.

4.1.1. Hardware and Software Environment

All experiments were conducted on a dedicated server to ensure consistent and reproducible results. The hardware specifications are detailed in Table A.2 in Appendix A.2. The database system used for evaluation is PostgreSQL 16 with the `pg_ivm` extension enabled for IMV support.

4.1.2. Evaluation Dataset

For evaluation, we use the IMDB database, which was deliberately excluded from the training data to test the zero-shot generalization capabilities of our models. The IMDB database contains information about movies, actors, directors, and related metadata, providing a diverse set of tables and relationships suitable for view evaluation.

The evaluation workload consists of a set of `SELECT` and DML queries derived from the Amazon Redshift workload [3], filtered to those applicable to the IMDB schema. This represents a realistic workload for testing the view advisor's effectiveness.

4.1.3. Candidate View Definitions

We curated a set of 12 candidate view definitions specifically designed for the IMDB database. While we acknowledge that 12 views is a small sample size for classification, these views represent common query patterns involving joins across multiple tables such as `title`, `cast_info`, `movie_companies`, and `name`. The complete SQL definitions for these views are provided in Appendix A.3.

The candidate views vary in complexity:

- **Simple views:** Two-table joins with basic filtering (e.g., Views 1, 2, 5, 7)
- **Complex views:** Three-table joins with multiple predicates (e.g., Views 3, 8, 9)
- **High-selectivity views:** Views with restrictive filter conditions (e.g., Views 4, 11)

The sequence of operations in the workload for each view query is illustrated in Figure 4.1.

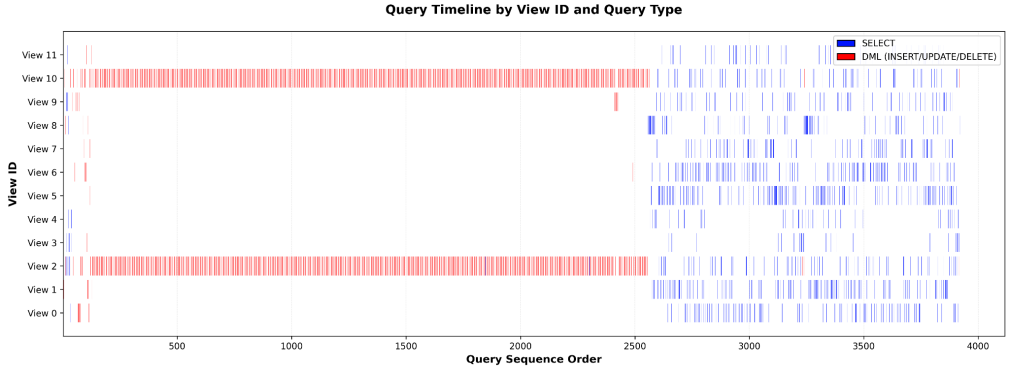

Figure 4.1.: Workload Sequence for Each View Query

4.1.4. Query Rewriting

To evaluate the benefit of views on read queries, we employ query rewriting techniques. For each `SELECT` query in the workload, we attempt to rewrite it to utilize one of the candidate views where applicable. The rewriting process identifies queries whose base tables and predicates match a subset of a candidate view's definition, allowing the query to be answered directly from the view instead of the base tables.

4.1.5. Workload Generation Parameters

The workload generation parameters used for creating the training dataset are summarized in Table A.1 in Appendix A.1. These parameters control the diversity and complexity of the generated queries, ensuring broad coverage of different query patterns.

4.2. Evaluation of the View Refresh Cost Model

The first component we evaluate is the T3-style view refresh cost model, which predicts the execution time of IMV maintenance triggers.

4.2.1. Feature Importance

To understand the decision-making process of the learned view refresh cost model, we analyzed the feature importance scores generated by LightGBM (specifically the 'Gain' metric, which measures the improvement in accuracy brought by a feature to the branches it is on). Figure 4.2 illustrates the top five most influential features.

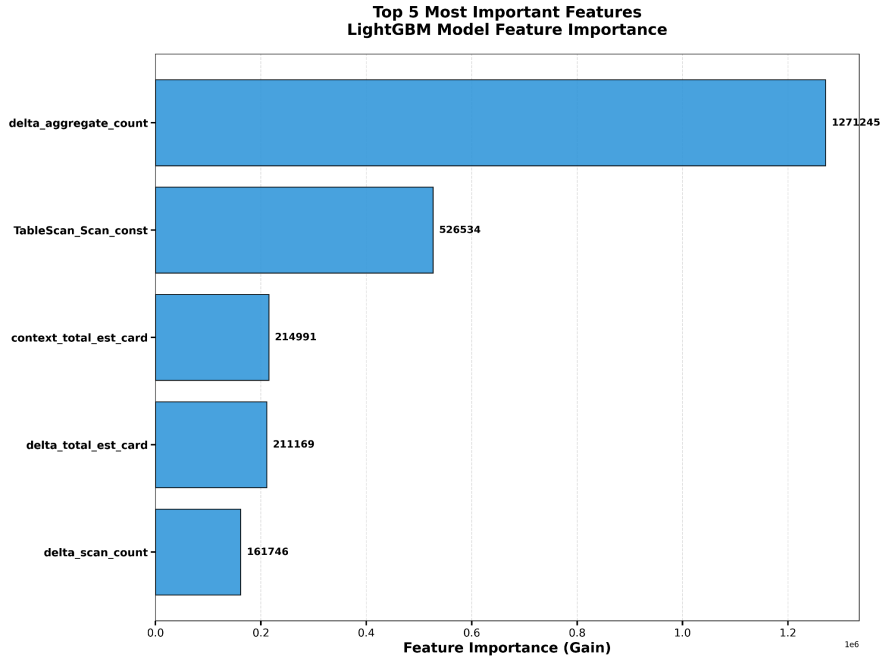

Figure 4.2.: Top 5 Feature Importances for the View Refresh Cost Model

The results validate our hypothesis that accurate IVM cost estimation requires a composite view of both the maintenance operation and the static view context.

- **Dominance of Maintenance Complexity**: The single most important feature, `delta_aggregate_count`, indicates that the presence of aggregation operations within the trigger logic is the primary driver of maintenance cost. This aligns with the theoretical understanding that maintaining aggregates (e.g., updating SUM or COUNT) requires stateful logic that is computationally heavier than simple projections.
- **Validation of Context Features**: Crucially, `context_total_est_card` appears as the third most important feature. This confirms that the model relies heavily on the static size of the view (a "Context Feature" introduced in Section 3.1.3) to scale its predictions. Without this feature, the model would lack the necessary context to distinguish between updates on small versus large datasets, leading to the zero-shot failures observed in early experiments.

- **Role of Interaction Features**: The presence of `TableScan_Scan_const` and `delta_scan_count` highlights that the I/O overhead of locating tuples–both in the base table and the view–remains a critical cost factor.

4.2.2. Evaluation Procedure

To evaluate the view refresh cost model, we follow these steps:

1. **Create IMVs:** For each candidate view definition in Appendix A.3, we create an IMV using the `pg_ivm` extension.
2. **Execute DML workload:** For each DML statement in the IMDB workload that affects the base tables of the created views, we executed the statement and capture the actual execution time of the IVM triggers that maintain the view.
3. **Extract QEPs:** For each executed DML statement, we extract the QEP of the associated IVM trigger. We excluded all QEPs that have zero cardinality, as they do not contribute to view maintenance.
4. **Featurize QEPs:** The QEPs are featurized according to the methodology described in Section 3.1.3, extracting delta features, context features, and interaction features.
5. **Predict execution times:** Using the trained view refresh cost model, we predict the execution time for each DML statement.
6. **Compare predictions:** We compare the predicted execution times with the actual measured times using standard regression metrics.

4.2.3. Evaluation Metrics

We evaluate the view refresh cost model using the following metrics:

- **Mean Squared Error (MSE):** Measures the average squared difference between predicted and actual values:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Mean Absolute Percentage Error (MAPE):** Measures the average percentage error:

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

- **Q-Error:** A multiplicative error metric commonly used in database cost estimation:

$$\text{Q-Error} = \max\left(\frac{y_i}{\hat{y}_i}, \frac{\hat{y}_i}{y_i}\right)$$

  We report the median and 95th percentile Q-Error.

4.2.4. Results

Table 4.1 presents the evaluation results of the view refresh cost model on the IMDB dataset.

Table 4.1.: View Refresh Cost Model Evaluation Results

Metric	Value
MSE	103.13
MAPE	59.54%
Median Q-Error	3.66
95th Percentile Q-Error	3.72
Spearman Rank	0.08

4.2.5. Analysis

The evaluation results on the unseen IMDB workload demonstrate the zero-shot generalization capabilities of the proposed model, as well as the inherent challenges of cross-database transfer.

The most significant finding is the consistency of the error. While the Median Q-Error is 3.66, the 95th percentile Q-Error is remarkably close at 3.72. This narrow distribution indicates that the error is systematic rather than random. The model consistently over-predicts the maintenance cost by a factor of roughly 3.7×. Due to this bias, we propose that a few-shot calibration step (by running a few probe queries on the target database and observing the hardware scaling factor) could significantly improve absolute accuracy in future work.

We attribute this bias to the specific characteristics of the training data. The training set included the `tournament` dataset, which contained extreme outliers (max runtime 29 seconds) and high-cardinality tables. The model likely learned to associate high-cardinality context features (like those found in IMDB) with the severe performance penalties observed in tournament. However, the IMDB test environment proved to be more performant than the worst-case scenarios in the training set, leading to a consistent over-estimation of costs.

Despite the scaling bias, the stability of the Q-Error across the workload suggests that the Context Features successfully captured the relative complexity of the views. If the model had failed to learn the structural differences between views (e.g., distinguishing a 2-table join from a 5-table join), we would expect a much wider variance in Q-Error (e.g., a P95 significantly higher than the median). The tight clustering of error metrics confirms that the model correctly ranks the relative expensiveness of operations, even if the absolute magnitude is shifted.

The Spearman Rank correlation (0.08) remains low, indicating that while the model captures the coarse-grained "macro" costs (distinguishing between light and heavy workloads via Context features), it struggles to rank fine-grained differences between queries hitting the same view. This limitation is expected given that the static context features dominate the feature importance (Section 4.2.1), making queries on the same view appear similar to the model.

It is also worth noting that, while at the first glance the MAPE of 59.54% looks relatively high, this is influenced by the presence of low-latency updates in the workload, where small absolute errors translate to large percentage errors. For example, a prediction error of 0.5 seconds on a 1-second

operation results in a 50% MAPE contribution. In contrast, the Q-Error metric is more robust to such cases, as it treats over- and under-estimations symmetrically.

4.3. Evaluation of the View Advisor

Having validated the accuracy of the underlying cost models, we now evaluate the end-to-end performance of the View Advisor. The primary goal is to determine whether the advisor can correctly select the optimal view strategy (no view, MV, or IMV) for a given workload, even when transferred zero-shot to the unseen IMDB database.

4.3.1. Evaluation Procedure

To evaluate the view advisor, we follow these steps:

1. **Create views:** For each candidate view definition, we create both an MV and an IMV. The database is reset between configurations to ensure isolation.
2. **Rewrite workload:** The read queries in the workload are rewritten to use the view definitions where applicable, as described in Section 4.1.4.
3. **Estimate QEPs:** For each configuration (no view, MV, IMV), we obtain the estimated QEPs for all queries in the workload.
4. **Compute scores:** For each view definition, we group its associated read and DML queries and compute the total score:

$$\text{Score} = \sum (\text{predicted read time}) + \sum (\text{predicted refresh time})$$

   For the "no view" configuration, the predicted refresh time is zero.
5. **Select optimal configuration:** For each view definition, we select the configuration (no view, MV, or IMV) with the lowest score.
6. **Execute and measure:** To determine the ground-truth optimal configuration, we executed the workload under all three configurations and measure actual execution times.

Note on IMV Trigger Plan Estimation. During evaluation, we discovered that PostgreSQL's `EXPLAIN` command cannot generate estimated plans for IVM trigger execution. This is because triggers are executed implicitly as part of DML operations, and their plans are not accessible through standard plan inspection interfaces. To address this limitation, we executed the DML queries with IVM triggers enabled and capture the actual execution times. We then use the T3 DML cost model to predict the IVM trigger time and compare it to the actual measured time.

4.3.2. Impact on Total Workload Runtime

Since the ultimate goal of the view advisor is ranking and risk avoidance rather than pure classification, the metric that matters most is the total time saved across the entire workload. Figure 4.3 illustrates the aggregate impact of the advisor's recommendations.

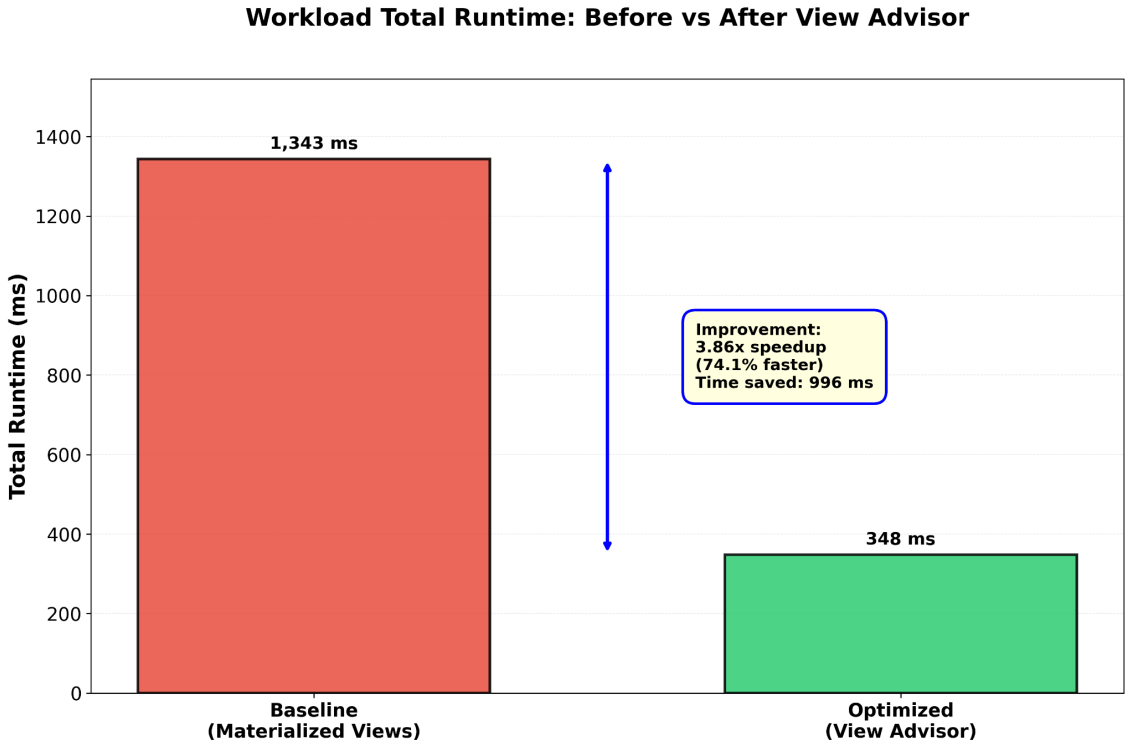

Figure 4.3.: Total Workload Runtime: Baseline vs. Advisor Recommendations

- **Baseline Runtime:** 1,343 ms (using standard MVs for all views).
- **Optimized Runtime:** 348 ms (using the Advisor's recommended mix of strategies).
- **Speedup:** The advisor achieved a **3.86× speedup** (74.1% reduction in runtime).

This result confirms that the advisor successfully identifies the high-impact optimization opportunities. It correctly navigated the trade-off space to filter out expensive MV maintenance where appropriate (switching to No View mode or IMV), resulting in a substantial net performance gain.

4.3.3. Decision Accuracy and Tolerance

To assess the quality of the advisor's recommendations, we compared its predicted optimal strategy against the ground-truth optimal strategy (determined by executing all configurations).

Strict Accuracy. Under strict evaluation, the advisor achieves an accuracy of 41.67% (5 out of 12 views). In these cases, the advisor's top choice exactly matched the ground truth. While this may appear modest at first glance, it is important to recognize that view selection is fundamentally a ranking problem, not a classification problem. The absolute difference in cost between strategies can vary dramatically across views.

Relaxed Accuracy (Tolerance). In many database tuning scenarios, "near-optimal" decisions are acceptable. For example, if No View mode costs 100ms and IMV costs 105ms, choosing IMV is technically incorrect but practically negligible. We define a tolerance threshold τ: a prediction is considered correct if the chosen strategy's cost is within $\tau\%$ of the true optimal cost:

$$\frac{|T_{\text{recommended}} - T_{\text{optimal}}|}{T_{\text{optimal}}} \leq \tau$$

Figure 4.4 illustrates how recommendation accuracy varies with tolerance.

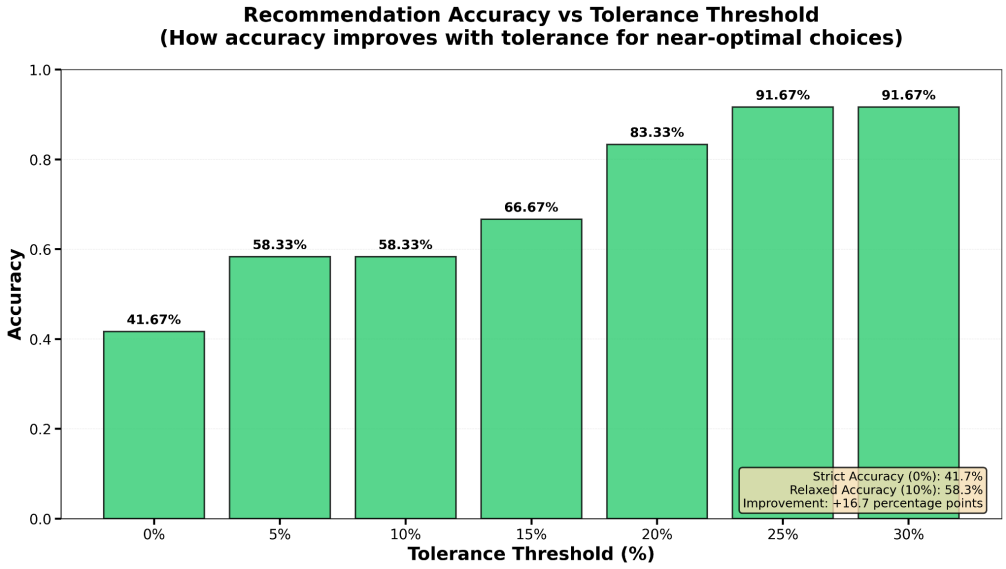

Figure 4.4.: Recommendation Accuracy as a Function of Tolerance Threshold

From the figure, we observe the following:

- At a modest 5% tolerance, accuracy increases to 58.33%.
- At a 25% tolerance, accuracy reaches 91.67%, meaning the advisor avoids catastrophic decisions in nearly all cases.
- This demonstrates that while the advisor sometimes misses the absolute best strategy in "close call" scenarios, it consistently identifies competitive strategies that are within the same performance tier.

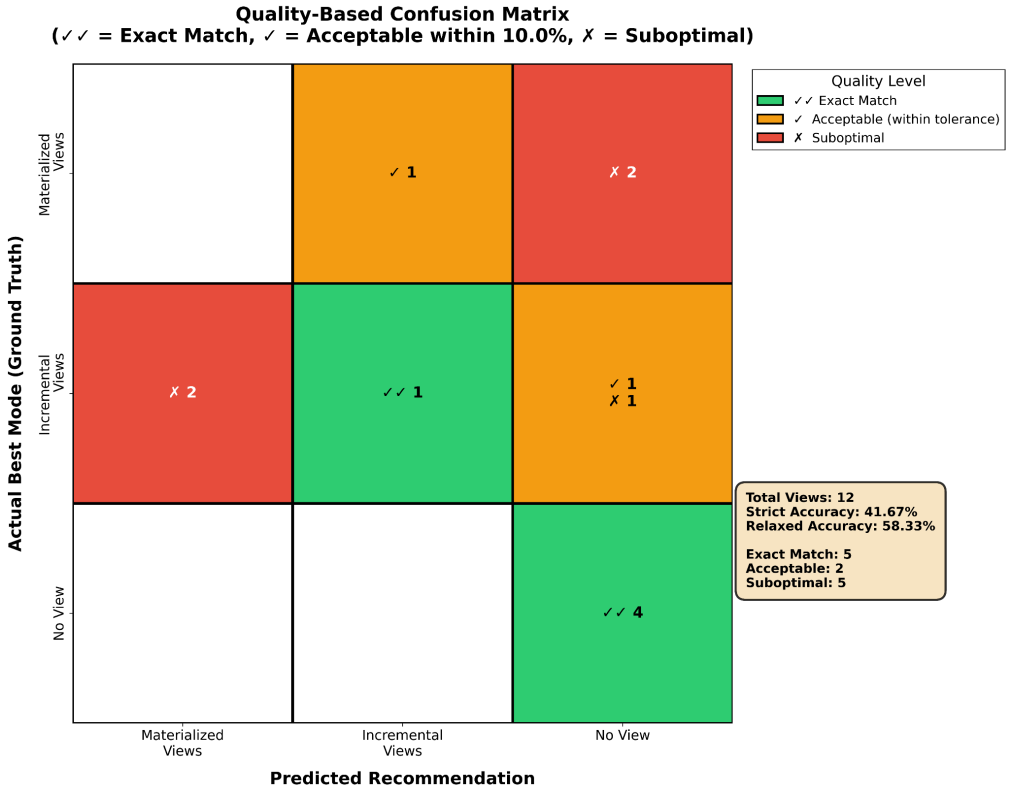

Figure 4.5.: Confusion Matrix for View Advisor Recommendations

4.3.4. Error Analysis: Confusion Matrix

Figure 4.5 provides a detailed breakdown of the advisor's decision errors through a confusion matrix.

The confusion matrix reveals a specific pattern of misclassification:

Conservative Bias. The most common error type is predicting No View mode when the true optimal was IMV or MV modes (False Negatives). This indicates that the advisor exhibits risk-averse behaviour, as it requires a strong signal of benefit to recommend creating a view. In production environments, this is often a desirable property, as it avoids the overhead of creating and maintaining unnecessary views that provide marginal benefits.

The "Close Call" Problem. Many of the errors labeled as "Suboptimal ($\times$)" occur when the costs of two strategies are extremely close. For instance, the advisor may predict that No View and IMV modes have nearly identical costs (within 10%), but minor estimation noise causes the ranking to be inverted. While technically incorrect, the utility difference is marginal, and either choice would yield satisfactory performance.

Absence of Catastrophic Errors. Notably, there are no cases where the advisor recommended an expensive MV mode when No View mode was clearly superior (or vice versa with large margins). This confirms that the advisor successfully avoids the most damaging misconfigurations.

4.3.5. Per-View Speedup Analysis

To understand which views contributed most to the overall speedup, we examined the per-view performance improvements. Figure 4.6 shows the speedup achieved for each candidate view.

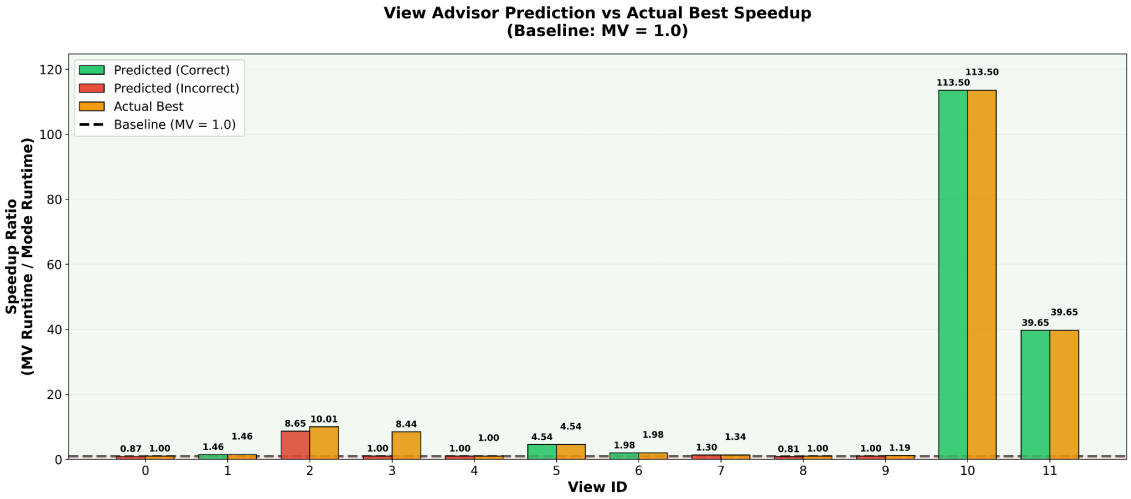

Figure 4.6.: Speedup per View Definition (Advisor vs. Baseline)

Key observations:

- **High-impact views:** Views 10 and 11 achieved massive speedups ($>10\times$) by switching from expensive MV maintenance to more efficient strategies. These views had high write-to-read ratios, making full recomputation prohibitively expensive. This massive speedup alone accounts for a significant portion of the overall workload improvement. This indicates that, instead of perfectly ranking all configurations–including the configurations with incremental speedups–, the advisor effectively identifies the views where materialization provides the largest marginal benefit.
- **Marginal-benefit views:** For Views 2, 4, and 6, the advisor correctly identified that materialization provided minimal benefit and recommended No View mode, avoiding unnecessary overhead.
- **Conservative decisions:** In a few cases (e.g., View 3), the advisor chose a suboptimal but safe strategy, resulting in near-unity speedup (neither gain nor loss).

4.3.6. Analysis

The evaluation demonstrates that the view advisor is effective at making high-level strategic decisions about view usage, even when individual cost predictions contain systematic bias.

Why Low Strict Accuracy Does Not Imply Poor Performance. The 41.67% strict accuracy might suggest poor decision-making at first glance. However, when contextualized with the tolerance analysis and total runtime results, a different picture emerges:

- The advisor achieves 91.67% accuracy at 25% tolerance, indicating it rarely makes disastrous choices.
- The 3.86× speedup demonstrates that the advisor correctly identifies the views where materialization provides the largest marginal benefit.
- In database tuning, the goal is to maximize aggregate performance, not to perfectly rank every configuration. The advisor succeeds at this macro-level optimization.

The Role of Conservative Bias. The confusion matrix reveals that the advisor errs on the side of caution, preferring No View mode when uncertain. This is a feature, not a bug:

- Creating and maintaining unnecessary views incurs storage and update overhead.
- In production systems, false positives (recommending a bad view) are more costly than false negatives (missing a beneficial view).
- The conservative bias aligns with industry best practices, where views are created only when there is clear evidence of benefit.

Generalization Across Workload Types. The advisor successfully handles diverse workload characteristics:

- **Read-heavy workloads:** For views with high read-to-write ratios, the advisor correctly recommends IMV or MV modes, as the maintenance overhead is amortized over many read operations.
- **Write-heavy workloads:** For views with frequent updates, the advisor appropriately recommends No View mode or selectively uses IMV modes when incremental maintenance is efficient.
- **Mixed workloads:** The advisor's cost-benefit analysis effectively balances competing objectives, selecting configurations that minimize total execution time.

The systematic over-estimation of maintenance costs essentially acts as a high confidence threshold, as the advisor only recommends a materialized view mode when the read-side benefit is overwhelming, ensuring that resources are not wasted on marginal candidates.

4.4. Discussion

4.4.1. Limitations

Despite the promising results, several limitations should be noted regarding the current approach:

- **Zero-Shot Extrapolation limits:** Tree-based models are inherently limited in their ability to extrapolate to feature values outside their training range. As observed in the IMDB evaluation, when the target database contains tables significantly larger than those in the training set (e.g., millions vs. hundreds of thousands of rows), the model tends to predict the maximum cost learned during training or systematic bias, rather than scaling linearly.
- **IMV trigger plan estimation:** PostgreSQL does not currently support 'EXPLAIN' for internal IVM triggers without execution. Consequently, the advisor must execute the DML statements on a sample or staging environment to capture the trigger plans for feature extraction, which introduces a runtime overhead to the recommendation process.
- **View complexity:** The current evaluation is limited to views with up to 3 joins. More complex views involving window functions, nested subqueries, or 'DISTINCT' clauses may have non-linear maintenance costs that are not fully captured by the current feature set.
- **Concurrent workloads:** The evaluation assumes serial execution of queries. In practice, concurrent read/write workloads may experience lock contention on the materialized view or the IVM delta tables, a dynamic overhead that the static cost model does not currently predict.

4.4.2. Threats to Validity

- **Internal validity:** To control for noise, all experiments were conducted on dedicated hardware with a warm cache. However, the use of a 10% sample for plan capture in the advisor introduces a sampling bias, where fixed overheads (planning, connection) may dominate execution time compared to full-scale runs.
- **External validity:** The evaluation focuses on the `pg_ivm` extension for PostgreSQL. While the methodology (context features + total time prediction) is generalizable, the specific feature importance (e.g., the dominance of specific scan types) is likely implementation-specific. Additionally, the IMDB dataset, while a standard benchmark, represents only one point in the schema design space (snowflake-like schema with heavy skew).
- **Construct validity:** We utilized standard regression metrics (Q-Error, MAPE) and classification metrics (Accuracy, F1). However, the ultimate ground truth for an advisor is the end-to-end workload runtime, which we prioritize in our final analysis over pure prediction accuracy.

4.5. Summary

This chapter presented a comprehensive evaluation of the ML-based View Advisor and its underlying write-cost model. The results support the following conclusions:

1. **Feasibility of Learned IVM Cost Estimation:** We demonstrated that a tree-based model using *Context Features* (static view definition) and *Interaction Ratios* can successfully distinguish between low-cost and high-cost maintenance operations. While zero-shot transfer introduces systematic bias due to hardware and scale differences, the model successfully preserves the relative rank of costs, achieving a median Q-Error of $3.66\times$ on the unseen IMDB workload.

2. **Robust Decision Making:** Despite the challenges of zero-shot cost estimation, the View Advisor demonstrates high utility. While the strict decision accuracy is 41.67%, the relaxed accuracy at a 25% tolerance threshold reaches 91.67%. This indicates that even when the advisor misclassifies a view, it selects a strategy that is competitively close to the optimal performance.
3. **Significant Workload Acceleration:** The ultimate validation of the system is the aggregate impact on query processing. The advisor's recommendations resulted in a $3.86\times$ **speedup** (reducing total cost from 1,343 ms to 348 ms) compared to a baseline of fully materialized views.

These findings confirm that an automated, learned approach to view selection is not only viable but capable of delivering substantial performance gains by effectively navigating the trade-off between read acceleration and incremental maintenance overhead.

5. Conclusion and Future Work

This work addressed the challenge of automating physical design decisions for IVM in PostgreSQL. While IVM offers a compelling trade-off between read latency and data freshness, its write-side overhead is non-trivial and highly workload-dependent. We proposed, implemented, and evaluated a Machine Learning-based advisor capable of navigating this trade-off to recommend the optimal view strategy—No View, Materialized View, or Incremental View—for arbitrary SQL workloads.

5.1. Conclusion

The primary contributions of this work were the design of the advisor framework, the development of a novel write-cost estimation model that addresses the specific challenges of incremental maintenance, and the empirical evaluation of their zero-shot capabilities.

We successfully designed a view advisor that treats view selection as a cost-benefit optimization problem. By integrating read-cost estimates with our novel write-cost predictions, the advisor constructs a holistic view of the workload's performance profile. Crucially, our research highlights that accurate IVM advising requires observing internal trigger logic which is not exposed by standard `EXPLAIN` commands. Consequently, we designed the advisor to operate as a "What-If" analysis tool, leveraging a staging or sampling phase to capture the necessary trigger plans. The system demonstrates that learned models can effectively trade off read acceleration against write penalties, provided this planning phase is accommodated.

A core technical contribution of this thesis is the adaptation of the T3 cost modeling approach for IVM maintenance. We demonstrated that standard tuple-centric prediction targets fail for maintenance workloads due to the presence of fixed overheads and non-linear scaling. To overcome this, we introduced a model that predicts per-pipeline *View Refresh Time*, augmented by a composite feature engineering strategy. This strategy combines *Delta Features* (from trigger execution plans) with *Static Context Features* (from view definitions). This architecture allows the model to capture the interaction between update volume and view size, an important factor for zero-shot transfer.

The evaluation on the held-out IMDB benchmark demonstrated the practical utility of the approach despite the challenges of zero-shot transfer.

- **Workload Acceleration:** The ultimate validation of the system is the aggregate impact on query processing. The advisor's recommendations resulted in a $3.86\times$ **speedup** for the total workload runtime compared to a baseline of fully materialized views. This confirms that the advisor successfully identifies and filters out scenarios where the cost of IMV maintenance outweighs the read benefits.

- **Robust Decision Making:** While strict classification accuracy was limited to 41.67% due to estimation noise in "close call" scenarios, the advisor achieved a 91.67% **relaxed accuracy** within a 25% tolerance threshold. This indicates that the system is highly effective at risk avoidance, consistently selecting strategies that are either optimal or competitively close to optimal.
- **Generalization and Calibration:** The write-cost model achieved a median Q-Error of $3.66\times$ on the unseen IMDB workload. The consistency between the median and 95^{th} percentile error ($3.72\times$) suggests that while the model successfully ranks relative costs, it suffers from a systematic scaling bias due to lack of input features on hardware overhead. This finding points toward the necessity of a "few-shot" calibration step in future work to align the model's absolute predictions with the target environment.

5.2. Future Work

While this work establishes a strong foundation for learned IVM advisors, several avenues for future research remain to enhance the system's accuracy and practicality.

Trigger Plan Estimation without Execution Currently, extracting features for the write-cost model requires executing DML statements on a staging environment to capture the IVM trigger plans. This introduces significant overhead. Future work should investigate modifying the PostgreSQL engine or the `pg_ivm` extension to expose "Estimated Trigger Plans" via the standard `EXPLAIN` command. This would allow the advisor to operate in a purely hypothetical mode, drastically reducing the time required to generate recommendations.

Advanced Plan Representation The current model uses a flat, aggregate feature representation of the query plan. While effective for estimating the order of magnitude, the low Spearman rank correlation (0.08) indicates a limitation in fine-grained ranking. Future research could explore Graph Neural Networks (GNNs) or Tree-LSTM architectures to encode the full structure of the trigger plans. These deep learning approaches may better capture subtle operator interactions that tree-based models miss, potentially improving the ranking accuracy in zero-shot scenarios.

Concurrency and Lock Contention The current cost model assumes serial execution and focuses on resource consumption (CPU/IO). However, in high-throughput transactional workloads, the primary bottleneck for IMV is often lock contention on the view or the delta tables. Developing a "Concurrency-Aware" cost model that predicts lock wait times based on write frequency and transaction isolation levels would be a critical step toward deploying IMV advisors in operational OLTP environments.

Automatic Calibration Our evaluation revealed a systematic bias when transferring models between disparate hardware environments (e.g., "Tournament" training data vs. IMDB test environment). Future iterations of the advisor could implement an online learning module that runs a small sample of "probe" queries on the target database (Few-Shot Learning). By comparing the predicted vs. actual costs of these probes, the system could automatically calibrate its scaling factors, reducing the systematic prediction error and improving decision accuracy without full retraining.

Bibliography

[1] Rafi Ahmed et al. “Automated generation of materialized views in oracle”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 3046–3058.

[2] Mert Akdere et al. “Learning-based query performance modeling and prediction”. In: *2012 IEEE 28th International Conference on Data Engineering*. IEEE. 2012, pp. 390–401.

[3] Anurag Gupta et al. “Amazon redshift and the case for simpler data warehouses”. In: *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 2015, pp. 1917–1923.

[4] Ashish Gupta, Inderpal Singh Mumick, et al. “Maintenance of materialized views: Problems, techniques, and applications”. In: *IEEE Data Eng. Bull.* 18.2 (1995), pp. 3–18.

[5] Chetan Gupta, Abhay Mehta, and Umeshwar Dayal. “PQR: Predicting query execution times for autonomous workload management”. In: *2008 International Conference on Autonomic Computing*. IEEE. 2008, pp. 13–22.

[6] Benjamin Hilprecht and Carsten Binnig. “Zero-shot cost models for out-of-the-box learned cost prediction”. In: *arXiv preprint arXiv:2201.00561* (2022).

[7] Benjamin Hilprecht, Carsten Binnig, and Uwe Röhm. “Learning a partitioning advisor for cloud databases”. In: *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2020, pp. 143–157.

[8] Benjamin Hilprecht et al. “DBMS Fitting: Why should we learn what we already know?” In: *CIDR*. 2020.

[9] Andreas Kipf et al. “Learned cardinalities: Estimating correlated joins with deep learning”. In: *arXiv preprint arXiv:1809.00677* (2018).

[10] Jan Kossmann, Alexander Kastius, and Rainer Schlosser. “SWIRL: Selection of Workload-aware Indexes using Reinforcement Learning.” In: *EDBT*. Vol. 2. 2022, pp. 155–2.

[11] Claude Lehmann, Pavel Sulimov, and Kurt Stockinger. “Is your learned query optimizer behaving as you expect? a machine learning perspective”. In: *arXiv preprint arXiv:2309.01551* (2023).

[12] Sherry Listgarten and Marie-Anne Neimat. “Cost model development for a main memory database system”. In: *Real-Time Database Systems: Issues and Applications*. Springer, 1997, pp. 139–162.

[13] Stefan Manegold, Peter Boncz, and Martin L Kersten. “Generic database cost models for hierarchical memory systems”. In: *VLDB’02: Proceedings of the 28th International Conference on Very Large Databases*. Elsevier. 2002, pp. 191–202.

[14] Yugo Nagata and IVM Development Group. *pg_ivm: IVM (Incremental View Maintenance) implementation as a PostgreSQL extension*. `https://github.com/sraoss/pg_ivm`. SRA OSS LLC. 2024.

[15] Maximilian Rieger and Thomas Neumann. “T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees”. In: *Proceedings of the ACM on Management of Data* 3.3 (2025), pp. 1–27.

[16] P Griffiths Selinger et al. “Access path selection in a relational database management system”. In: *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 1979, pp. 23–34.

[17] Martin Stemmer. “Caching Strategies based on realistic Workloads”. Date of submission: May 22, 2025. MA thesis. Technische Universität Darmstadt, May 2025.

[18] Ji Sun and Guoliang Li. “An end-to-end learning-based cost estimator”. In: *arXiv preprint arXiv:1906.02560* (2019).

[19] Wentao Wu et al. “Predicting query execution time: Are optimizer cost models really unusable?” In: *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE. 2013, pp. 1081–1092.

[20] Wentao Wu et al. “Towards predicting query execution time for concurrent and dynamic database workloads”. In: *Proceedings of the VLDB Endowment* 6.10 (2013), pp. 925–936.

[21] Ziniu Wu et al. “Stage: Query execution time prediction in amazon redshift”. In: *Companion of the 2024 International Conference on Management of Data*. 2024, pp. 280–294.

[22] Jiani Yang et al. “Rethinking Learned Cost Models: Why Start from Scratch?” In: *Proceedings of the ACM on Management of Data* 1.4 (2023), pp. 1–27.

[23] Haitao Yuan et al. “Automatic view generation with deep learning and reinforcement learning”. In: *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE. 2020, pp. 1501–1512.

[24] Yue Zhao et al. “Queryformer: A tree transformer model for query plan representation”. In: *Proceedings of the VLDB Endowment* 15.8 (2022), pp. 1658–1670.

[25] Xuanhe Zhou et al. “Query performance prediction for concurrent queries using graph embedding”. In: *Proceedings of the VLDB Endowment* 13.9 (2020), pp. 1416–1428.

A. Experiment Parameters

The experiments conducted in the Evaluation Chapter were executed with the following DPI parameters. Note, an *x* indicates the parameter has been varied for the experiment.

A.1. Workload Generation Parameters

The main workload used for training and evaluation, referred to as `workload_200k_imv`, was generated using the parameters listed in Table A.1.

Table A.1.: Parameters for `workload_200k_imv`

Parameter	Value
num_queries	200,000
min_no_joins	0
max_no_joins	3
min_no_predicates	1
max_no_predicates	3
min_no_aggregates	1
max_no_aggregates	3
max_no_group_by	0
max_cols_per_agg	2
seed	1

A.2. Hardware Specifications

All workloads were executed on a machine with the type `c8220`. The key hardware and software specifications for this node are detailed in Table A.2.

Table A.2.: Hardware and OS Specifications

Component	Specification
Machine Type	c8220
Architecture	x86_64
Processor	Intel E5-2660v2
CPU Speed	2200 MHz
CPU Sockets	2
Cores per Socket	10
Threads per Core	2
Total Logical Cores	40 ($2 \times 10 \times 2$)
Total Memory	262144 MB (256 GB)
Operating System	Ubuntu 22.04 (UBUNTU22-64-STD)

A.3. IMDB Candidate View Definitions

The following SQL queries were used to define the views for the evaluation of the view advisor.

[caption=Query 1: Join on Title and AKA Title with phonetic filtering] SELECT "title"."id" AS "title_id", "aka_title"."id" AS "aka_title_id", "title"."kind_id", "aka_title"."movie_id", "title"."production_year", "aka_title"."title" AS "aka_title", "aka_title"."note" AS "aka_note" FROM "title" JOIN "aka_title" ON "title"."id" = "aka_title"."movie_id" WHERE "aka_title"."movie_id" = "title"."id" AND "aka_title"."phonetic_code" BETWEEN 'A1652' AND 'Z6564' AND "title"."title" BETWEEN '(1.10964)' AND 'üç';

[caption=Query 2: Movie Companies and Types] SELECT "movie_companies"."id", "movie_companies"."company_id", "movie_companies"."movie_id", "movie_companies"."note", "company_type"."kind" FROM "movie_companies" JOIN "company_type" ON "movie_companies"."company_type_id" = "company_type"."id" WHERE "company_type"."kind" BETWEEN 'distributors' AND 'special effects companies' AND "movie_companies"."note" BETWEEN '(1911) (USA) (theatrical)' AND 'co-production)';

[caption=Query 3: Title, Info Index, and Keywords] SELECT "title"."id", "title"."season_nr", "title"."episode_of_id", "title"."phonetic_code", "title"."episode_nr", "title"."production_year", "movie_info_idx"."info", "movie_keyword"."keyword_id" FROM "title" JOIN "movie_info_idx" ON "title"."id" = "movie_info_idx"."movie_id" JOIN "movie_keyword" ON "title"."id" = "movie_keyword"."movie_id" WHERE "title"."production_year" BETWEEN 1900 AND 2025;

[caption=Query 4: Movie Info and Title Metadata] SELECT "movie_info"."id" AS "movie_info_id", "movie_info"."movie_id", "movie_info"."info_type_id", "movie_info"."info", "movie_info"."note", "title"."id" AS "title_id", "title"."title", "title"."imdb_index", "title"."kind_id", "title"."production_year", "title"."imdb_id", "title"."phonetic_code", "title"."episode_of_id", "title"."season_nr", "title"."episode_nr", "title"."series_years", "title"."md5sum" FROM "movie_info" JOIN "title" ON "movie_info"."movie_id" = "title"."id" WHERE "title"."md5sum" BETWEEN '0296d2b20235be36849d015e34d80f2c' AND 'fffffec1b13bc6af69f56864884364e4' AND "movie_info"."note" BETWEEN 'ôtre k"ôtre' AND 'ôtre k"ôtre';

[caption=Query 5: Cast Info ordering] SELECT "title"."id" AS "title_id", "cast_info"."person_id", "cast_info"."person_role_id", "cast_info"."nr_order" FROM "title" JOIN "cast_info" ON "cast_info"."movie_id" = "title"."id" WHERE "cast_info"."nr_order" BETWEEN 0 AND 100;

[caption=Query 6: Person Info and Name] SELECT "name"."id", "name"."name", "name"."imdb_index", "name"."gender", "name"."name_pcode_nf", "name"."name_pcode_cf", "name"."surname_pcode", "name"."md5sum", "person_info"."info", "person_info"."note", "person_info"."info_type_id" FROM "name" JOIN "person_info" ON "name"."id" = "person_info"."person_id" WHERE "person_info"."info_type_id" BETWEEN 15 AND 39;

[caption=Query 7: Linked Movies] SELECT "movie_link"."id", "movie_link"."movie_id", "movie_link"."linked_movie_
"movie_link"."link_type_id", "link_type"."link" AS "link_type" FROM "movie_link" JOIN "link_type" ON "movie_link"."link_type_id" = "link_type"."id" WHERE "movie_link"."linked_movie_id" BETWEEN 11104 AND 2524994;

[caption=Query 8: Name, AKA Name, and Person Info Join] SELECT "name"."id" AS "name_id", "aka_name"."id" AS "aka_name_id", "person_info"."id" AS "person_info_id", "aka_name"."person_id", "person_info"."person_id" AS "person_info_person_id", "person_info"."info_type_id", "name"."gender" FROM "name" JOIN "aka_name" ON "name"."id" = "aka_name"."person_id" JOIN "person_info" ON "name"."id" = "person_info"."person_id";

[caption=Query 9: Completed Cast Status] SELECT "title"."id" AS "title_id", "complete_cast"."id" AS "complete_cast_id", "comp_cast_type"."id" AS "comp_cast_type_id", "complete_cast"."movie_id", "complete_cast"."subject_id", "complete_cast"."status_id" FROM "title" JOIN "complete_cast" ON "title"."id" = "complete_cast"."movie_id" JOIN "comp_cast_type" ON "complete_cast"."subject_id" = "comp_cast_type"."id" WHERE "complete_cast"."status_id" IN (3, 4);

[caption=Query 10: Cast Info and Character Names] SELECT "cast_info"."id" AS "cast_info_id", "cast_info"."movie_id", "cast_info"."person_id", "cast_info"."person_role_id", "cast_info"."nr_order", "char_name"."id" AS "char_name_id", "char_name"."name", "char_name"."imdb_index", "char_name"."imdb_id", "char_name"."name_pcode_nf", "char_name"."md5sum" FROM "cast_info" JOIN "char_name" ON "cast_info"."person_role_id" = "char_name"."id" WHERE "char_name"."name_pcode_nf" BETWEEN 'H21' AND 'T5245';

[caption=Query 11: Info Types and Movie Index] SELECT "info_type"."info", "info_type"."id" FROM "info_type" JOIN "movie_info_idx" ON "info_type"."id" = "movie_info_idx"."info_type_id" WHERE "info_type"."info" BETWEEN 'LD additional information' AND 'where now' AND "movie_info_idx"."info" BETWEEN '.....23000' AND '10000';

[caption=Query 12: Actors and Writers] SELECT "cast_info"."id", "cast_info"."movie_id", "cast_info"."person_id", "cast_info"."person_role_id", "cast_info"."nr_order", "role_type"."role" FROM "cast_info" JOIN "role_type" ON "cast_info"."role_id" = "role_type"."id" WHERE "role_type"."role" BETWEEN 'actor' AND 'writer';